Introducing Quantile Regression

From Conditional Quantiles to the Regression Model

Tatjana Kecojević

2026-06-18

Table of contents

1 Learning Objectives

By the end of this session you will be able to:

- Explain how quantile regression extends OLS to the full conditional distribution
- Describe the check function and its role in quantile regression estimation
- Read and run `rq()` to fit quantile regression models in R
- Interpret quantile regression coefficients at specific values of
- Compare quantile regression and OLS estimates and understand when they diverge

2 Introduction

In the previous session, we introduced the concept of **conditional quantiles**: the idea that outcomes such as wages, health scores, or house prices are not equally distributed across a population, and that the shape of the distribution may itself depend on individual characteristics.

We ended by asking: how can we model conditional quantiles when we have many explanatory variables and a continuous predictor?

The answer is **quantile regression**.

Rather than modelling how explanatory variables shift the conditional mean, which is what OLS does, quantile regression models how they shift different parts of the conditional distribution. This allows us to examine questions that OLS cannot answer, such as:

- Is the return to education larger for low earners or high earners?
- Does an intervention reduce inequality, or does it benefit some groups more than others?
- Are extreme outcomes, the very healthy, the very wealthy, the most deprived, driven by different factors than average outcomes?

This session introduces the quantile regression model, explains the logic behind its estimation, and shows how to fit and interpret it in R using the `quantreg` package (Koenker 2005).

3 The Key Idea

Recall that ordinary least squares regression models the conditional mean:

$$E(Y \mid X) = X'\beta$$

Here, β is a single vector of coefficients that summarises the relationship between predictors and the **average** outcome. The model gives us one line through the data.

Quantile regression replaces the conditional mean with the **conditional quantile**:

$$Q_\tau(Y \mid X) = X'\beta(\tau)$$

Two things change:

1. The left-hand side is now $Q_\tau(Y \mid X)$: the τ-th conditional quantile of Y given X, rather than the conditional mean.
2. The coefficients $\beta(\tau)$ now **depend on** τ.

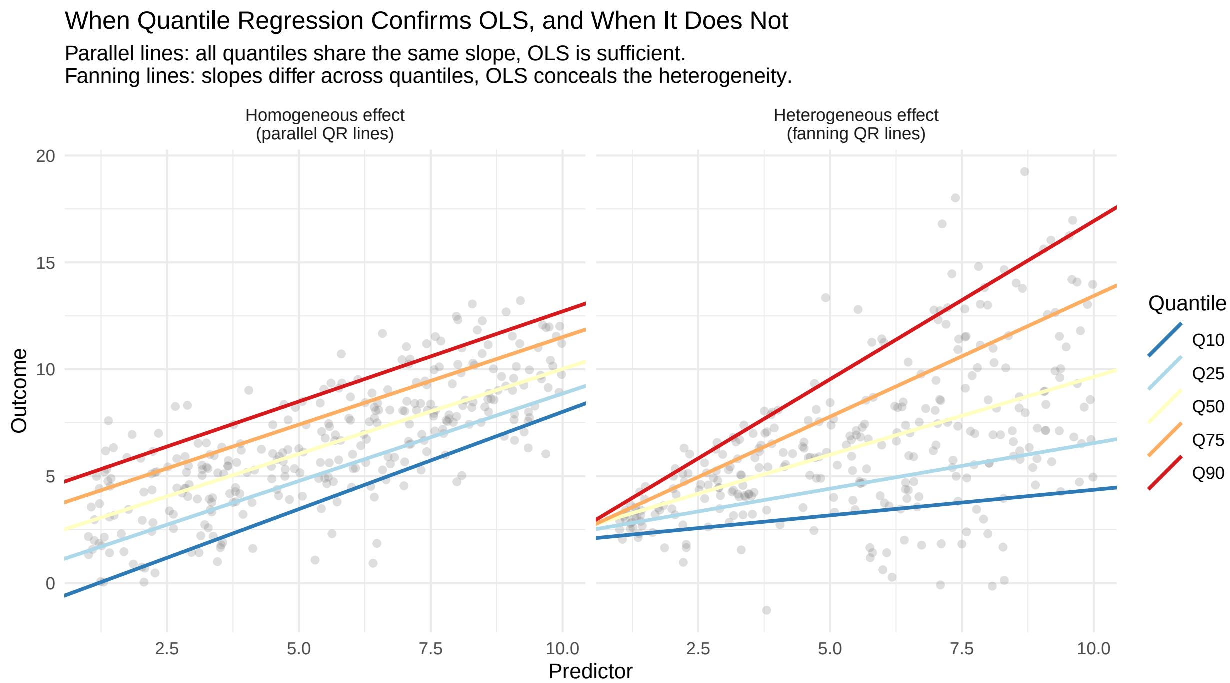
This second point is crucial. In OLS, one set of coefficients describes the relationship between predictors and the outcome. In quantile regression, **each quantile has its own set of coefficients**, and those coefficients can, and often do, differ.

τ	What is estimated
0.10	Conditional 10th percentile: the low end of the distribution
0.25	Conditional 25th percentile
0.50	Conditional median: the centre of the distribution
0.75	Conditional 75th percentile
0.90	Conditional 90th percentile: the high end of the distribution

If the effect of a predictor were the same at every quantile, all coefficient sets would be identical and quantile regression would simply confirm what OLS already tells us. When they differ, as they often do, we learn something important about **heterogeneity** in the relationship.

The figure below makes this concrete. Both panels show the same underlying mean relationship, but the left panel has constant variance while the right panel has variance that grows with the predictor. In the left panel, the quantile lines are parallel, all quantiles have the same slope, and OLS is telling you the full story. In the right panel, the lines fan out, lower quantiles have a shallow slope while upper quantiles have a steep slope. OLS averages over this variation and misses it entirely.

```r
n <- 300
x <- runif(n, 1, 10)

y_hom <- 2 + 0.8 * x + rnorm(n, 0, 2)
y_het <- 2 + 0.8 * x + rnorm(n, 0, 0.5 * x)

df_both <- bind_rows(
  tibble(x = x, y = y_hom, scenario = "Homogeneous effect\n(parallel QR lines)"),
  tibble(x = x, y = y_het, scenario = "Heterogeneous effect\n(fanning QR lines)")
) |>
  mutate(scenario = factor(scenario, levels = c(
    "Homogeneous effect\n(parallel QR lines)",
    "Heterogeneous effect\n(fanning QR lines)"
  )))

qr_df <- map_dfr(
  levels(df_both$scenario),
  \(sc) {
    d <- filter(df_both, scenario == sc)
    map_dfr(taus, \(t) {
      fit <- rq(y ~ x, data = d, tau = t)
      tibble(
        scenario  = factor(sc, levels = levels(df_both$scenario)),
        tau       = factor(paste0("Q", t * 100), levels = paste0("Q", taus * 100)),
        intercept = coef(fit)[1],
        slope     = coef(fit)[2]
      )
    })
  }
)

plot_het <- ggplot(df_both, aes(x = x, y = y)) +
  geom_point(alpha = 0.25, colour = "grey50") +
  geom_abline(
    data = qr_df,
    aes(intercept = intercept, slope = slope, colour = tau),
    linewidth = 0.9
  ) +
  scale_colour_brewer(palette = "RdYlBu", name = "Quantile", direction = -1) +
  facet_wrap(~scenario) +
  labs(
    title    = "When Quantile Regression Confirms OLS, and When It Does Not",
    subtitle = "Parallel lines: all quantiles share the same slope, OLS is sufficient.\nFanning lines:
    x        = "Predictor",
    y        = "Outcome"
  )

plot_het
```




```
ggsave(
  "figures/homogeneous_vs_heterogeneous.png",
  plot   = plot_het,
  width  = 9,
  height = 5
)
```

OLS vs Quantile Regression at a Glance

	OLS	Quantile Regression
Estimates	Conditional mean	Conditional quantile
Coefficients	One set	One set per quantile τ
Lines through the data	One	One per τ (a family of lines)
Sensitive to outliers	Yes	Less so
Key question	What is the average effect?	Does the effect vary across the distribution?

4 How Is It Estimated? The Loss Function

To understand how quantile regression is estimated, it helps to first think about OLS, and then see how quantile regression generalises that idea.

4.1 OLS and Squared Loss

OLS estimates β by finding the coefficients that minimise the sum of **squared** residuals:

$$\min_{\beta} \sum_{i=1}^{n} (y_i - x_i'\beta)^2$$

Squaring the residuals penalises large errors disproportionately. The result is a line that passes through the centre of the conditional distribution, the mean, and is pulled toward any extreme values.

4.2 The Median and Absolute Loss

Now suppose we minimised the sum of **absolute** residuals instead:

$$\min_{\beta} \sum_{i=1}^{n} |y_i - x_i'\beta|$$

This is the **least absolute deviations** estimator. It is more robust to outliers because it does not square the residuals, extreme values are penalised proportionally rather than quadratically. The resulting line passes through the **conditional median** rather than the conditional mean.

This establishes an important principle: *the criterion we minimise determines which part of the distribution we estimate.*

A simple illustration. Consider five households with annual food expenditures of 100, 150, 200, 250, and 800 francs. Their mean is 300 (pulled upward by the outlier at 800) and their median is 200.

The figure below shows the total absolute deviation as a function of a candidate summary value, the number we would be choosing if we minimised absolute deviations. The curve reaches its minimum at exactly 200, the median. The mean (300) sits noticeably further up the curve.

```
households    <- c(100, 150, 200, 250, 800)
candidates    <- seq(50, 850, by = 1)
total_abs_dev <- sapply(candidates, \(m) sum(abs(households - m)))

plot_lad <- tibble(candidate = candidates, total_abs_dev = total_abs_dev) |>
  ggplot(aes(x = candidate, y = total_abs_dev)) +
  geom_line(colour = "steelblue", linewidth = 1) +
  geom_vline(xintercept = median(households), colour = "#2ca25f",
             linetype = "dashed", linewidth = 1) +
  geom_vline(xintercept = mean(households),   colour = "tomato",
             linetype = "dashed", linewidth = 1) +
  annotate("text", x = median(households) + 20, y = 1550,
           label = paste0("Median = ", median(households)),
           colour = "#2ca25f", hjust = 0, size = 3.8) +
  annotate("text", x = mean(households) + 20,   y = 1450,
           label = paste0("Mean = ",   mean(households)),
           colour = "tomato",  hjust = 0, size = 3.8) +
  labs(
    title    = "The Median Minimises Total Absolute Deviation",
    subtitle = "Five households: 100, 150, 200, 250, 800",
    x        = "Candidate summary value",
    y        = "Total absolute deviation"
  )

plot_lad

ggsave(
  "figures/lad_median.png",
  plot   = plot_lad,
  width  = 7,
  height = 4
)
```

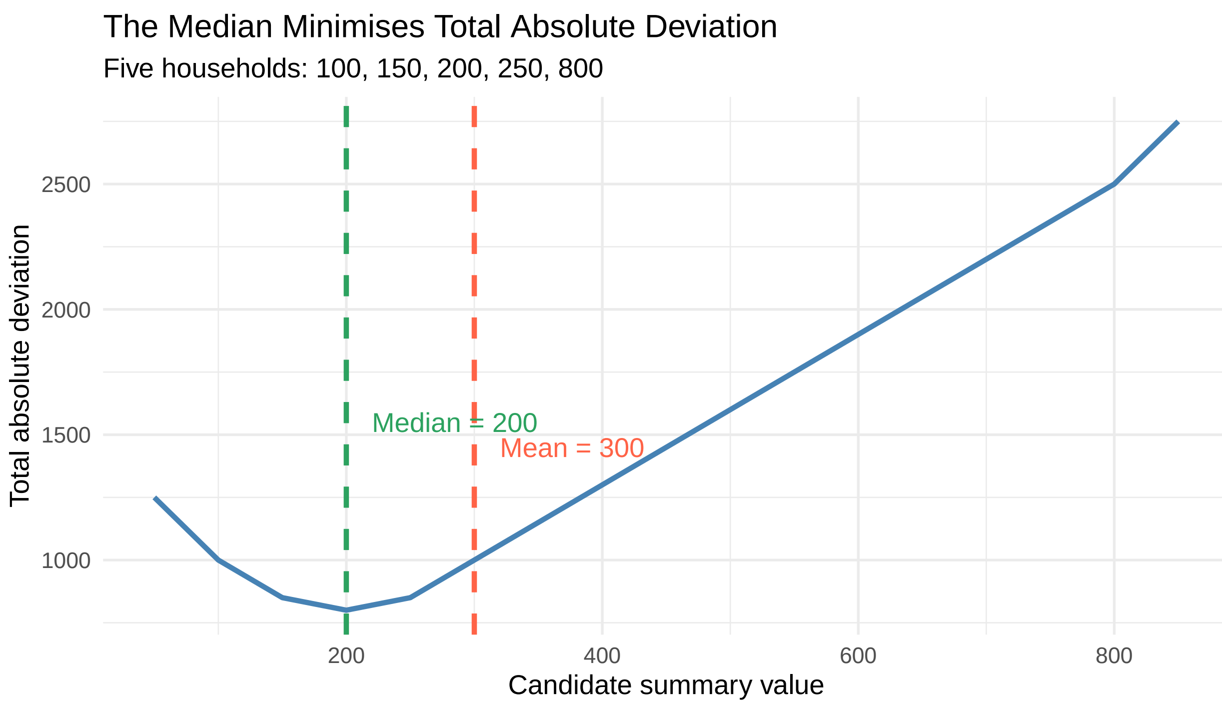


Figure 1

The outlier at 800 pulls the mean far from where the majority of observations sit. The median, by minimising absolute rather than squared deviations, is unaffected. This robustness property carries forward into quantile regression.

4.3 The Check Function

Quantile regression generalises the absolute loss function by making it **asymmetric**. The resulting function is known as the check function or tilted absolute loss:

$$\rho_\tau(u) = u \cdot (\tau - \mathbf{1}(u < 0))$$

where $u = y_i - x_i'\beta(\tau)$ is the residual.

In plain terms:

- When $u > 0$ (the observation lies **above** the fitted line), the penalty is $u \times \tau$.
- When $u < 0$ (the observation lies **below** the fitted line), the penalty is $|u| \times (1 - \tau)$.

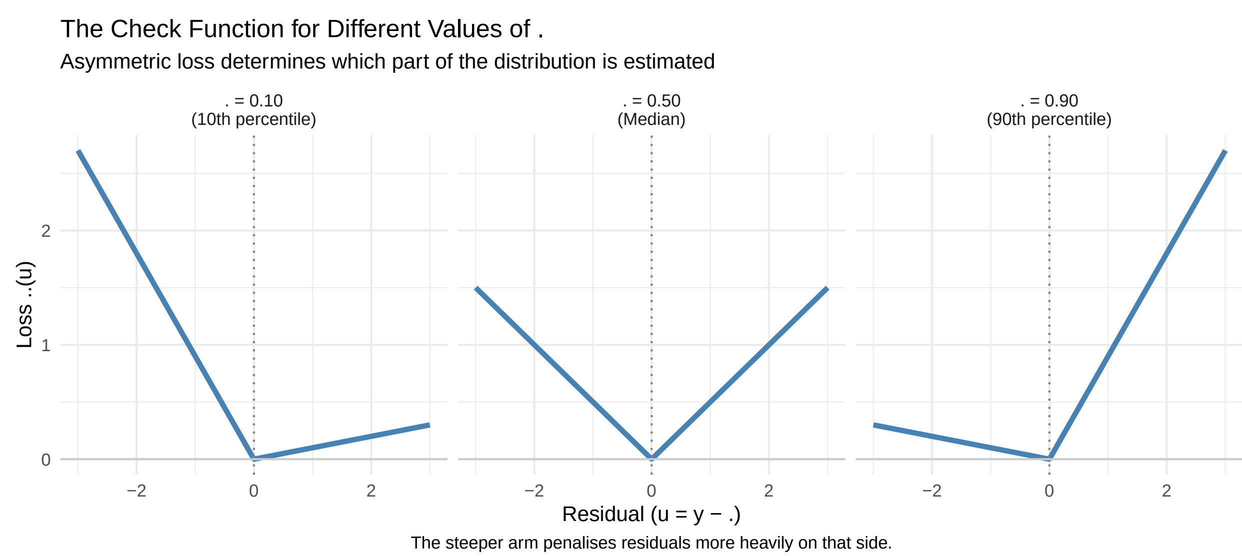
The asymmetry shifts depending on τ:

- At $\tau = \mathbf{0.10}$, negative residuals (overpredictions) are penalised heavily → the fitted line is pushed down to capture the lower tail.
- At $\tau = \mathbf{0.50}$, positive and negative residuals are penalised equally → this recovers the median.
- At $\tau = \mathbf{0.90}$, positive residuals (underpredictions) are penalised heavily → the fitted line is pushed up to capture the upper tail.

The figure below shows the shape of the check function for three values of τ.

```
check_fn <- function(u, tau) u * (tau - as.numeric(u < 0))

u_vals    <- seq(-3, 3, length.out = 500)
```

```r
tau_vals   <- c(0.10, 0.50, 0.90)
tau_labels <- c("  = 0.10\n(10th percentile)",
                "  = 0.50\n(Median)",
                "  = 0.90\n(90th percentile)")

loss_df <- map_dfr(seq_along(tau_vals), \(i) {
  tibble(
    u    = u_vals,
    loss = check_fn(u_vals, tau_vals[i]),
    tau  = factor(tau_labels[i], levels = tau_labels)
  )
})

plot_check <- loss_df |>
  ggplot(aes(x = u, y = loss)) +
  geom_line(linewidth = 1.3, colour = "steelblue") +
  geom_vline(xintercept = 0, linetype = "dotted", colour = "grey50") +
  geom_hline(yintercept = 0, colour = "grey80") +
  facet_wrap(~tau) +
  labs(
    title    = "The Check Function for Different Values of  ",
    subtitle = "Asymmetric loss determines which part of the distribution is estimated",
    x        = "Residual (u = y − ŷ)",
    y        = "Loss  (u)",
    caption  = "The steeper arm penalises residuals more heavily on that side."
  ) +
  theme(plot.caption = element_text(hjust = 0.5))

plot_check
```




```r
ggsave(
  "figures/check_function.png",
  plot   = plot_check,
  width  = 9,
  height = 4
)
```

Notice how the slope on each side of zero shifts with τ. At $\tau = 0.10$, the left arm is steep and the right arm is shallow, observations above the line are penalised lightly, while observations below it are penalised heavily, pulling the line down. At $\tau = 0.90$, this is reversed.

> **!** Key Insight
>
> Changing τ changes the relative penalty assigned to over-prediction and under-prediction.
> This causes the fitted regression line to move toward different parts of the conditional distribution.

4.4 The Check Function in Action: A Numerical Example

The formula becomes much clearer when applied to real numbers. The following example uses seven observations drawn from a standard normal distribution, and asks: which value of ξ minimises the check function sum for $\tau = 0.25$?

$\min_\xi \sum_{i=1}^{n} \rho_{0.25}(y_i - \xi)$

We first try a **wrong guess** ($\xi = 0.674$, the fifth observation), then the **optimal value** ($\xi = -0.612$, the true 25th percentile). For each observation, the table shows the residual $u = y_i - \xi$, the indicator $\mathbf{1}(u < 0)$, which simply flags whether the observation falls *below* the candidate line, and the resulting loss $\rho_{0.25}(u)$.

Notice what the indicator column does: it is 1 for every observation that lies *below* the candidate value, and 0 for every observation that lies *above* it. At the optimal $\xi = -0.612$, exactly **one out of seven observations** falls below, which is approximately $1/7 \approx 14\%$, consistent with the 25th percentile of this particular small sample.

The indicator function $\mathbf{1}(u < 0)$ is the engine of the asymmetry. It automatically detects which side of the fitted line each observation lies on, and the check function assigns the appropriate weight (τ or $1 - \tau$) accordingly.

> Intuition Without the Algebra
>
> You do not need to calculate the check function by hand. The key insight is:
>
> > Quantile regression finds the regression surface where approximately $\tau \times$ **100%** of observations lie below the fitted values.
>
> At $\tau = 0.25$, roughly 25% of observations lie below the fitted line.
> At $\tau = 0.75$, roughly 75% of observations lie below the fitted line.
> The asymmetric loss function is the mathematical machinery that achieves this.

5 Visual Intuition: The Engel Data

The best way to understand what quantile regression does is to see it in a picture. We use the **Engel dataset** from the `quantreg` package, one of the most widely used examples in the quantile regression literature (Koenker and Bassett 1978).

The dataset contains 235 observations of Belgian working-class households recorded in 1857 by the statistician Ernst Engel. Each row records annual household income and annual food expenditure. Engel's observation that the proportion of income spent on food declines as income rises became known as Engel's Law.

What makes this dataset particularly useful is a prominent feature of the data: **heteroscedasticity**. The spread of food expenditure grows as income increases. Wealthier households differ more from one another than poorer households do.

Table 3

```r
check_fn <- function(u, tau) u * (tau - as.numeric(u < 0))

y7  <- c(1.622, -0.161, -0.461, -0.612, 0.674, 0.711, -0.865)
tau_ex <- 0.25

build_check_table <- function(y, xi, tau) {
  u <- y - xi
  tibble(
    `y `          = round(y, 3),
    ` `           = xi,
    `u = y  -  ` = round(u, 3),
    `1(u < 0)`    = as.integer(u < 0),
    ` . (u)`  = round(check_fn(u, tau), 4)
  )
}

cat("Wrong guess    = 0.674  →  Sum =",
    round(sum(check_fn(y7 - 0.674, tau_ex)), 4), "\n")
```

```
Wrong guess    = 0.674  →  Sum = 3.8425
```

```r
cat("Optimal        = -0.612 →  Sum =",
    round(sum(check_fn(y7 - -0.612, tau_ex)), 4), "\n\n")
```

```
Optimal        = -0.612 →  Sum = 1.551
```

```r
cat("--- Wrong guess (  = 0.674) ---\n")
```

```
--- Wrong guess (  = 0.674) ---
```

```r
print(build_check_table(y7, 0.674, tau_ex))
```

```
# A tibble: 7 x 5
      y        `u = y  -  ` `1(u < 0)` ` . (u)`
   <dbl> <dbl>        <dbl>      <int>      <dbl>
1  1.62  0.674        0.948          0      0.237
2 -0.161 0.674       -0.835          1      0.626
3 -0.461 0.674       -1.14           1      0.851
4 -0.612 0.674       -1.29           1      0.964
5  0.674 0.674        0              0      0
6  0.711 0.674        0.037          0      0.0092
7 -0.865 0.674       -1.54           1      1.15
```

```r
cat("\n--- Optimal (  = -0.612, the true 25th percentile) ---\n")
```

```

--- Optimal (  = -0.612, the true 25th percentile) ---
```

```r
print(build_check_table(y7, -0.612, tau_ex))
```

```
# A tibble: 7 x 5
      y         `u = y  -  ` `1(u < 0)` ` . (u)`
   <dbl>  <dbl>        <dbl>      <int>      <dbl>
1  1.62  -0.612        2.23           0      0.558
2 -0.161 -0.612        0.451          0      0.113
3 -0.461 -0.612        0.151          0      0.0377
4 -0.612 -0.612        0              0      0
5  0.674 -0.612        1.29           0      0.322
6  0.711 -0.612        1.32           0      0.331
7 -0.865 -0.612       -0.253          1      0.190
```

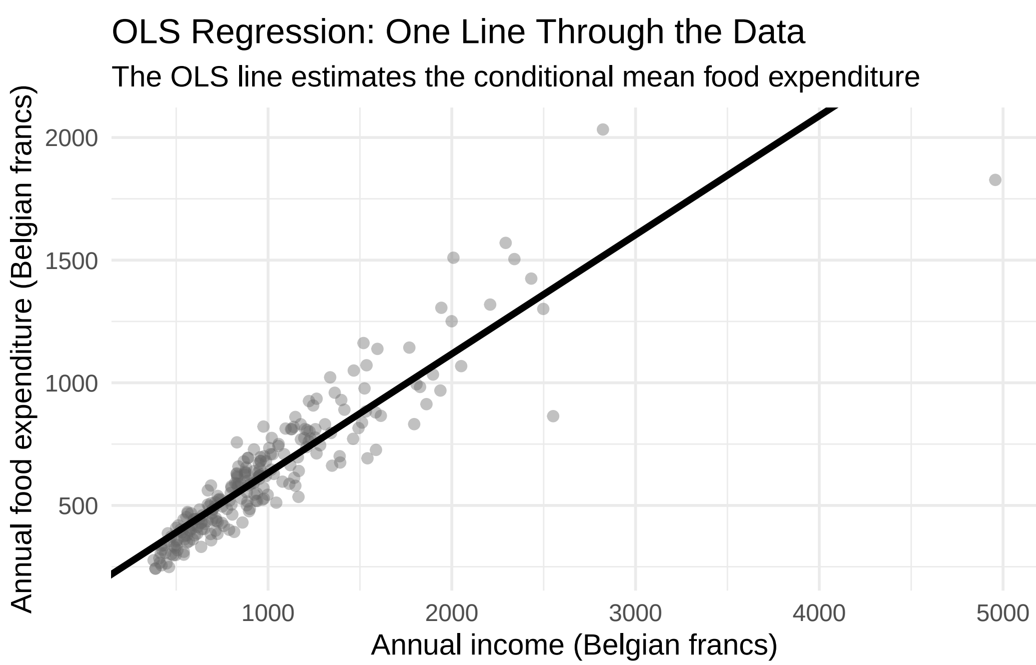
```r
data(engel, package = "quantreg")
glimpse(engel)
```

```
Rows: 235
Columns: 2
$ income  <dbl> 420.1577, 541.4117, 901.1575, 639.0802, 750.8756, 945.7989, 82~
$ foodexp <dbl> 255.8394, 310.9587, 485.6800, 402.9974, 495.5608, 633.7978, 63~
```

5.1 OLS: One Line Through the Data

We begin with OLS. It gives us one line, the conditional mean food expenditure at each income level.

```r
ols_engel <- lm(foodexp ~ income, data = engel)

plot_ols <- engel |>
  ggplot(aes(x = income, y = foodexp)) +
  geom_point(alpha = 0.40, colour = "grey40") +
  geom_abline(
    intercept = coef(ols_engel)[1],
    slope     = coef(ols_engel)[2],
    linewidth  = 1.2
  ) +
  labs(
    title    = "OLS Regression: One Line Through the Data",
    subtitle = "The OLS line estimates the conditional mean food expenditure",
    x        = "Annual income (Belgian francs)",
    y        = "Annual food expenditure (Belgian francs)"
  )

plot_ols
```




```r
ggsave(
  "figures/engel_ols.png",
```

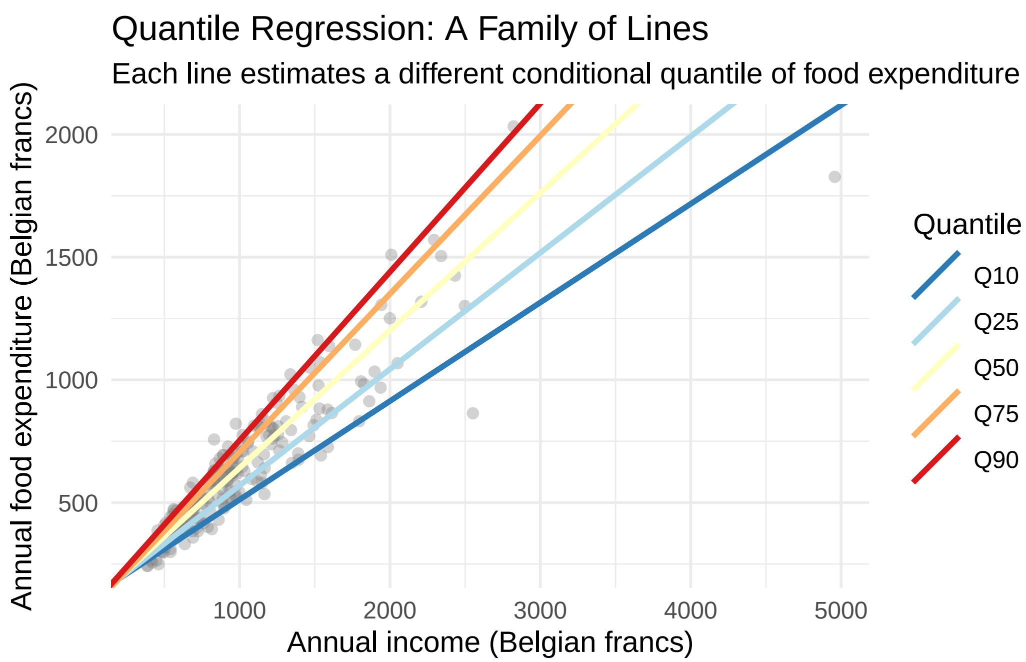
```r
  plot   = plot_ols,
  width  = 7,
  height = 5
)
```

The OLS line provides a reasonable summary of the centre of the data. But it cannot capture the spreading pattern that is clearly visible: at higher income levels, the points fan out substantially. A single line drawn through the centre misses what is happening at the edges.

5.2 Quantile Regression: A Family of Lines

Now we fit quantile regression at five quantile levels: $\tau = 0.10, 0.25, 0.50, 0.75, 0.90$. Each level produces its own line.

```r
qr_lines <- map_dfr(taus, \(t) {
  fit <- rq(foodexp ~ income, data = engel, tau = t)
  tibble(
    tau       = factor(paste0("Q", t * 100), levels = paste0("Q", taus * 100)),
    intercept = coef(fit)[1],
    slope     = coef(fit)[2]
  )
})

plot_qr <- engel |>
  ggplot(aes(x = income, y = foodexp)) +
  geom_point(alpha = 0.35, colour = "grey50") +
  geom_abline(
    data = qr_lines,
    aes(intercept = intercept, slope = slope, colour = tau),
    linewidth = 1
  ) +
  scale_colour_brewer(
    palette   = "RdYlBu",
    name      = "Quantile",
    direction = -1
  ) +
  labs(
    title    = "Quantile Regression: A Family of Lines",
    subtitle = "Each line estimates a different conditional quantile of food expenditure",
    x        = "Annual income (Belgian francs)",
    y        = "Annual food expenditure (Belgian francs)"
  )

plot_qr
```




```r
ggsave(
  "figures/engel_qr_fan.png",
  plot   = plot_qr,
  width  = 7,
  height = 5
)
```

The spreading of the lines reveals what OLS concealed. At lower income levels, the quantile lines cluster together, households with similar incomes have relatively similar food expenditures. At higher income levels, the lines diverge, the relationship becomes more variable.

This is heteroscedasticity made visible. The single OLS line averages over all of this variation. Quantile regression reveals its structure.

5.3 A Practical Note: Can Quantile Lines Cross?

A natural question when looking at a family of quantile lines is: **can they cross?** If the line for $\tau = 0.75$ drops below the line for $\tau = 0.25$ at some value of x, that would be contradictory, it would imply that the 75th percentile is smaller than the 25th percentile, which is impossible by definition.

In theory, quantiles are always ordered: for any fixed x, $Q_{\tau_1}(Y \mid x) \leq Q_{\tau_2}(Y \mid x)$ whenever $\tau_1 < \tau_2$. The conditional quantile function is non-decreasing in τ.

In practice, with finite samples, *estimated* quantile regression lines can cross, particularly when the sample is very small, when extreme quantiles are fitted (τ close to 0 or 1), or when a flexible nonlinear model is used. This is a finite-sample artefact, not a theoretical failure.

The following example uses just seven observations (a bivariate dataset used by Koenker and Bassett, 1982) to make this concrete. With so few data points and such extreme quantiles as $\tau = 0.05$ and $\tau = 0.95$, the estimated lines are unstable, and crossing becomes possible at the extremes of the predictor range.

```r
df7 <- tibble(
  x = c(1, 2, 3, 5, 6, 8, 9),
  y = c(2, 1, 5, 2, 8, 6, 9)
)
```

```r
xbar7 <- mean(df7$x)

taus_tiny <- c(0.05, 0.25, 0.50, 0.75, 0.95)

qr7 <- map_dfr(taus_tiny, \(t) {
  fit <- rq(y ~ x, data = df7, tau = t)
  tibble(
    tau       = factor(paste0("Q", t * 100), levels = paste0("Q", taus_tiny * 100)),
    intercept = coef(fit)[1],
    slope     = coef(fit)[2]
  )
})

plot_crossing <- ggplot(df7, aes(x, y)) +
  geom_point(size = 3, colour = "grey30") +
  geom_abline(
    data = qr7,
    aes(intercept = intercept, slope = slope, colour = tau),
    linewidth = 1
  ) +
  geom_vline(xintercept = xbar7, linetype = "dashed", colour = "grey50") +
  annotate("text", x = xbar7 + 0.25, y = 1.0,
           label = paste0("\u0078\u0305 = ", round(xbar7, 2)),
           hjust = 0, size = 3.5, colour = "grey40") +
  scale_colour_brewer(palette = "RdYlBu", name = "Quantile", direction = -1) +
  scale_x_continuous(breaks = 1:9) +
  labs(
    title    = "Regression Quantile Lines: Seven-Observation Example",
    subtitle = "The vertical dashed line at x̄ traces the empirical quantile function at that point",
    x = "x", y = "y"
  )

plot_crossing

ggsave(
  "figures/seven_obs_qr.png",
  plot   = plot_crossing,
  width  = 7,
  height = 5
)
```

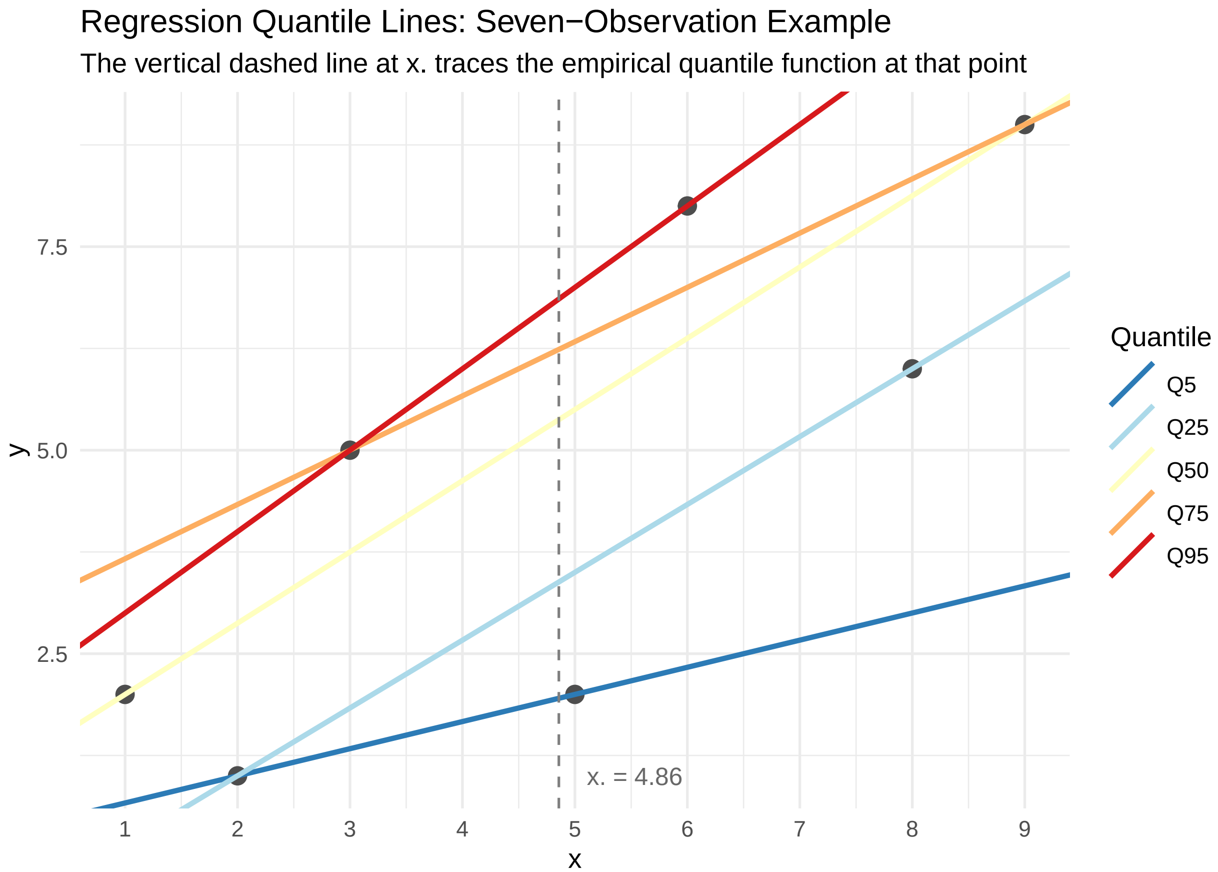


Figure 2

Notice that some lines come very close to crossing at the left edge of the plot. Now look at what happens when we read off the quantile line values *at the single point* $\bar{x} = 4.86$. Those values, one per τ, form the **empirical quantile function**, and it is guaranteed to be non-decreasing (Koenker 2005):

```
tau_fine  <- seq(0.01, 0.99, by = 0.01)
q_at_xbar <- map_dbl(tau_fine, \(t) {
  fit <- rq(y ~ x, data = df7, tau = t)
  predict(fit, newdata = data.frame(x = xbar7))
})

plot_stepfn <- tibble(tau = tau_fine, q = q_at_xbar) |>
  ggplot(aes(x = tau, y = q)) +
  geom_step(colour = "steelblue", linewidth = 1.2) +
  scale_x_continuous(
    breaks = c(0, 0.25, 0.50, 0.75, 1),
    labels = c("0", "0.25\n(Q1)", "0.50\n(Median)", "0.75\n(Q3)", "1")
  ) +
  labs(
    title    = "The Empirical Quantile Function Q(  | x̄)",
     subtitle = "Values of all quantile lines evaluated at x̄ = 4.86, always a non-decreasing step functi
    x        = " ",
    y        = "Q(  | x̄)"
  )
```

```r
plot_stepfn

ggsave(
  "figures/empirical_quantile_fn.png",
  plot   = plot_stepfn,
  width  = 7,
  height = 4
)
```

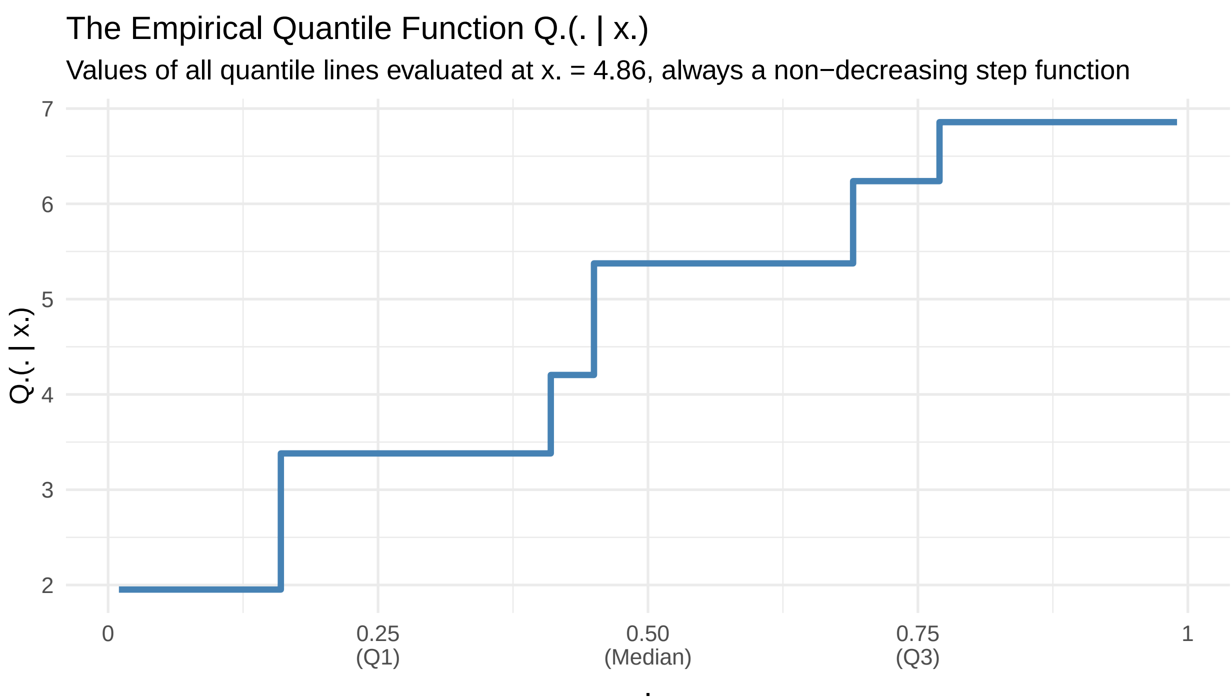


Figure 3

The step function is non-decreasing by construction: the jumps occur at values of τ where the fitted quantile moves from one observation to the next. The number of steps equals the number of distinct observations ($n = 7$ here), and the height of each step reflects the spacing between consecutive order statistics at $\bar{x}$.

ℹ The Crossing Issue in Practice

For the datasets used in this workshop (CPS1988 wages, $n = 28{,}155$) the crossing problem is negligible with linear quantile regression at moderate quantile levels (say, $\tau \in [0.05, 0.95]$). It becomes a concern mainly with:

- Very small samples
- Extreme quantiles ($\tau < 0.05$ or $\tau > 0.95$)
- Nonparametric or highly flexible quantile models

Methods exist to explicitly prevent crossing (Bondell, Reich & Wang, 2010; He, 1997), but for applied work with large survey data, it rarely arises.

❗ Important

Hands-On Activity: Finding the Quantile by Hand
Before running rq(), open the accompanying spreadsheet QR_CheckFunction.xlsx. It contains a small dataset and a live calculation of the check function sum for a given value of ξ and τ.

1) Try changing ξ manually and watch the sum respond. Can you find the value that minimises it? What happens when you change τ?
2) When you have found the minimum by hand, rq() in the next section will find it in a single line of code, for any dataset, at any quantile, instantly.

6 Introducing `rq()`

Quantile regression in R is implemented in the `quantreg` package through the `rq()` function. Its syntax is deliberately similar to `lm()`:

```
# Median regression (  = 0.5)
fit_median <- rq(foodexp ~ income, data = engel, tau = 0.5)
summary(fit_median)
```

```

Call: rq(formula = foodexp ~ income, tau = 0.5, data = engel)

tau: [1] 0.5

Coefficients:
            coefficients lower bd  upper bd
(Intercept)  81.48225     53.25915 114.01156
income        0.56018      0.48702   0.60199
```

To fit models at multiple quantile levels simultaneously, pass a vector to `tau`:

```
# Multiple quantiles at once
fit_multi <- rq(
  foodexp ~ income,
  data = engel,
  tau  = c(0.10, 0.25, 0.50, 0.75, 0.90)
)
```

The result is an object of class `rqs`. Calling `summary()` on it returns a separate set of coefficients and standard errors for each value of τ:

```
summary(fit_multi)
```

```

Call: rq(formula = foodexp ~ income, tau = c(0.1, 0.25, 0.5, 0.75,
    0.9), data = engel)

tau: [1] 0.1

Coefficients:
            coefficients lower bd  upper bd
(Intercept) 110.14157     79.88753 146.18875
income        0.40177      0.34210   0.45079

Call: rq(formula = foodexp ~ income, tau = c(0.1, 0.25, 0.5, 0.75,
```

```
    0.9), data = engel)

tau: [1] 0.25

Coefficients:
            coefficients lower bd  upper bd
(Intercept)  95.48354     73.78608 120.09847
income        0.47410      0.42033   0.49433

Call: rq(formula = foodexp ~ income, tau = c(0.1, 0.25, 0.5, 0.75,
    0.9), data = engel)

tau: [1] 0.5

Coefficients:
            coefficients lower bd  upper bd
(Intercept)  81.48225     53.25915 114.01156
income        0.56018      0.48702   0.60199

Call: rq(formula = foodexp ~ income, tau = c(0.1, 0.25, 0.5, 0.75,
    0.9), data = engel)

tau: [1] 0.75

Coefficients:
            coefficients lower bd  upper bd
(Intercept)  62.39659     32.74488 107.31362
income        0.64401      0.58016   0.69041

Call: rq(formula = foodexp ~ income, tau = c(0.1, 0.25, 0.5, 0.75,
    0.9), data = engel)

tau: [1] 0.9

Coefficients:
            coefficients lower bd  upper bd
(Intercept)  67.35087     37.11802 103.17399
income        0.68630      0.64937   0.74223
```

i Standard Errors in `rq()`

By default, `summary(rq())` reports standard errors using the **rank inversion** method. For applied work, bootstrapped standard errors are often preferred because they make fewer distributional assumptions. These can be requested with `se = "boot"`. We return to inference in detail in Session 04.

7 Interpreting Quantile Regression Coefficients

Interpreting quantile regression coefficients follows the same logic as OLS, with one important qualification: **every interpretation is anchored to a specific quantile**.

For a model of the form:

$$Q_\tau(\text{foodexp} \mid \text{income}) = \beta_0(\tau) + \beta_1(\tau) \times \text{income}$$

the slope coefficient $\beta_1(\tau)$ is interpreted as:

> The expected increase in food expenditure associated with a one-unit increase in income, **for households at the -th quantile of the food expenditure distribution**, conditional on income.

The following table extracts the estimated income slope at each quantile level.

```r
coef_df <- map_dfr(taus, \(t) {
  fit <- rq(foodexp ~ income, data = engel, tau = t)
  tibble(
    ` `        = t,
    `Quantile` = paste0(t * 100, "th percentile"),
    `Slope`    = round(coef(fit)["income"], 4)
  )
})

coef_df
```

```
# A tibble: 5 x 3
       Quantile        Slope
  <dbl> <chr>           <dbl>
1  0.1  10th percentile 0.402
2  0.25 25th percentile 0.474
3  0.5  50th percentile 0.560
4  0.75 75th percentile 0.644
5  0.9  90th percentile 0.686
```

Reading across the quantiles, the slope increases steadily from the bottom to the top of the distribution. Among households at the 10th quantile of food expenditure (low spenders relative to their income), a 100-franc increase in income is associated with an increase of approximately 40.2 francs in food expenditure. Among households at the 90th quantile (high spenders relative to their income), the same income increase is associated with approximately 68.6 francs.

The increasing pattern of slopes tells us that income has a stronger association with food expenditure among households who already spend heavily on food, a form of heterogeneity that is invisible to OLS.

The Interpretation Template

> "At the -th quantile of [**outcome**], a one-unit increase in [**predictor**] is associated with a change of [**coefficient**] units, holding other variables constant."

The phrase *at the -th quantile* is not optional, it is what distinguishes quantile regression from OLS, and what makes the interpretation meaningful.

8 Comparing Quantile Regression and OLS

8.1   = 0.5 and the Conditional Mean

When   = 0.50, quantile regression estimates the conditional **median**, not the conditional mean. For symmetric distributions, the mean and median coincide. For skewed distributions, they can diverge.

```r
ols_slope    <- coef(lm(foodexp ~ income, data = engel))["income"]
median_slope <- coef(rq(foodexp ~ income, data = engel, tau = 0.5))["income"]

tibble(
  Method   = c("OLS  (conditional mean)", "QR   = 0.50  (conditional median)"),
```

```
  `Slope` = round(c(ols_slope, median_slope), 4)
)
```

```
# A tibble: 2 x 2
  Method                            Slope
  <chr>                             <dbl>
1 OLS  (conditional mean)           0.485
2 QR   = 0.50  (conditional median) 0.560
```

The estimates are close here, but not identical. Food expenditure is modestly right-skewed, which pulls the OLS estimate slightly upward.

8.2 When Do the Methods Disagree?

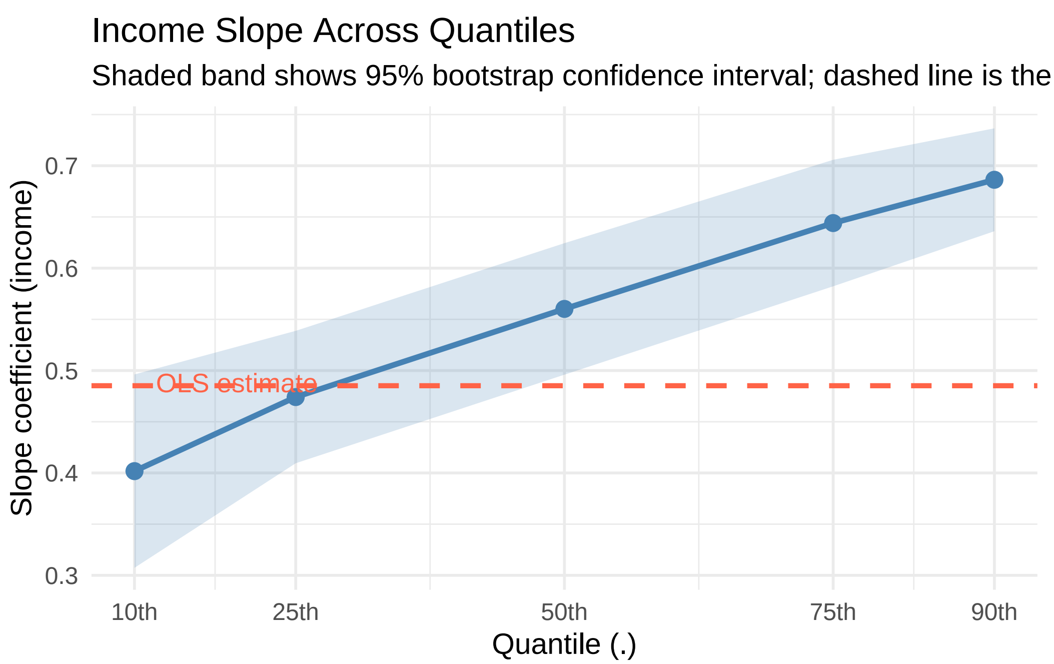
OLS and quantile regression will agree when the conditional distribution of the outcome is roughly symmetric and homoscedastic, in other words, when the mean is an adequate summary.

They will disagree, often substantially, in three situations that arise frequently in social science research:

1. **Skewed outcomes.** Wages, income, and house prices are right-skewed. The OLS estimate is influenced by the upper tail; quantile regression at the median is not.
2. **Heteroscedasticity.** When the variance of the outcome increases with a predictor (as in the Engel data), the slopes at different quantiles will differ systematically.
3. **Tail effects.** When the research question concerns extreme outcomes: the highest earners, the most deprived households, the best or worst-performing schools; quantile regression at high or low values is directly informative, whereas OLS averages over all of them.

```
coef_plot_df <- map_dfr(taus, \(t) {
  fit <- rq(foodexp ~ income, data = engel, tau = t)
  s   <- summary(fit, se = "boot")
  cm  <- s$coefficients
  tibble(
    tau   = t,
    est   = cm["income", "Value"],
    lower = cm["income", "Value"] - 1.96 * cm["income", "Std. Error"],
    upper = cm["income", "Value"] + 1.96 * cm["income", "Std. Error"]
  )
})

plot_coef <- ggplot(coef_plot_df, aes(x = tau, y = est)) +
  geom_ribbon(aes(ymin = lower, ymax = upper), alpha = 0.20, fill = "steelblue") +
  geom_line(colour = "steelblue", linewidth = 1) +
  geom_point(colour = "steelblue", size = 2.5) +
  geom_hline(
    yintercept = ols_slope,
    linetype   = "dashed",
    colour     = "tomato",
    linewidth  = 0.9
  ) +
  annotate(
    "text", x = 0.12, y = ols_slope + 0.003,
    label = "OLS estimate", colour = "tomato", size = 3.5, hjust = 0
  ) +
  scale_x_continuous(
    breaks = taus,
```

```
    labels = paste0(taus * 100, "th")
  ) +
  labs(
    title    = "Income Slope Across Quantiles",
    subtitle = "Shaded band shows 95% bootstrap confidence interval; dashed line is the OLS estimate",
    x        = "Quantile ( )",
    y        = "Slope coefficient (income)"
  )

plot_coef
```




```
ggsave(
  "figures/engel_coef_plot.png",
  plot   = plot_coef,
  width  = 7,
  height = 5
)
```

This coefficient plot is a key diagnostic in quantile regression analysis. If the slope were the same at every quantile, the blue line would be flat and would coincide with the OLS estimate (dashed red). Instead, the slope increases steadily, confirming that the relationship between income and food expenditure is systematically stronger among high-spending households.

> **!** Take-Home Message
>
> Quantile regression does not replace OLS, it complements it.
> If you fit quantile regression across several values of   and the coefficients are stable, OLS is giving you an adequate picture.
> If the coefficients vary substantially across quantiles, OLS is concealing important heterogeneity. Quantile regression reveals structure in the data that a single mean-based estimate would miss.

9 What Comes Next

This session introduced the quantile regression model using a deliberately simple dataset: one outcome, one predictor, a clear heteroscedastic pattern.

In the sessions that follow, we move to a richer applied example. Using the **CPS1988** dataset, a cross-section of US wages from the Current Population Survey, we will investigate:

- Does the return to an additional year of education differ between low and high earners?
- Is the wage gap between ethnic groups uniform across the distribution, or concentrated at particular quantiles?
- How do we assess uncertainty in quantile regression estimates?

These questions require both the tools introduced here and the inferential methods developed in Session 04.

10 References

Koenker, Roger. 2005. *Quantile Regression*. Cambridge University Press.

Koenker, Roger, and Gilbert Bassett. 1978. "Regression Quantiles." *Econometrica* 46 (1): 33–50.